

Attentional Branching Flow Matching

A Self-Calibrating Generative Set Framework

via Information-Theoretic Hierarchical Routing in Continuous Flow Matching

Research Concept — Complete Framework Draft

Central claim: The data supervises itself. ABFM asks the data how many modes it contains, where they are, and when to separate them — and the data answers through its own geometric structure. No external labels. No fixed K . No manual thresholds.

1. Motivation and Problem Statement

1.1 The Mode Averaging Problem

Standard flow matching learns a single vector field $u(x,t)$ that transports samples from a Gaussian source p_0 to a target data distribution p_1 . This works well when p_1 is unimodal. When p_1 is multimodal — containing multiple distinct clusters or families — the single vector field must simultaneously serve all modes. It averages across them, producing samples that sit between modes, belonging clearly to none. This is the mode averaging problem.

The failure is fundamental, not incidental. One vector field cannot faithfully represent multiple distinct transport geometries simultaneously. It will always compromise.

1.2 Why Existing Approaches Are Insufficient

Current solutions each require external knowledge the practitioner often does not have:

- Separate models per mode: requires knowing how many modes exist and which samples belong to each. Expensive, supervised, does not scale.

- Conditional flow matching: requires explicit conditioning labels. Supervised by definition.
- Mixture-of-Experts Flow Matching (MoE-FM, 2025): fixes K experts before training. K is a manual hyperparameter. Routing is a separate gating network, not intrinsic to the flow. Returns one sample per inference call. Cannot adapt K to data complexity.

All of these require the practitioner to impose structure on the data from outside. ABFM inverts this: the data reveals its own structure from inside.

1.3 The Core Insight

A sample's trajectory through the flow carries all the information needed to discover which mode it belongs to. As time t progresses from 0 to 1, samples drift from undifferentiated Gaussian noise toward their respective target modes. This drift is a geometric signal. ABFM is a framework that reads that signal and uses it to route samples to the correct branch — without any external supervision.

2. Framework Overview

Attentional Branching Flow Matching (ABFM) learns a binary tree of vector fields in one unified space. Each node in the tree is a single-head attention routing unit. Samples start at the root — Gaussian noise at $t=0$ — and traverse the tree, making routing decisions at each node guided by their own current position in the flow. A self-calibrating information-theoretic criterion detects genuine multimodality at each node and triggers splits only when mathematically warranted. Branches that capture no real structure are pruned. The output is a generative set: a structured, diverse collection of samples — one per surviving leaf — covering all meaningful modes the tree discovered.

ABFM is a strict generalization of standard flow matching. When the target distribution is unimodal, the information-theoretic criterion never fires, the tree reduces to a single path, and ABFM degenerates exactly to standard flow matching. No special case. No discontinuity.

3. Architecture

3.1 Shared Backbone

A shared MLP processes every sample at every node. Input is (x_t, t) — the sample's current position and timestep concatenated. Output is a query vector q representing where this sample is and how far along its trajectory it has traveled. All nodes share this backbone. Heavy computation happens once per step regardless of tree depth. Computational cost scales $O(d)$ with depth, not $O(2^d)$ with branch count.

$$q = \text{MLP}(x_t, t) \quad \text{where } x_t = t \cdot x_1 + (1-t) \cdot x_0$$

For structured inputs such as molecules or proteins, the MLP backbone can be replaced with a GNN, enabling relational reasoning over atomic structure before routing. For the proof of concept, a plain MLP with two hidden layers of 64 units and tanh activation is sufficient.

3.2 Node Architecture — Single-Head Attention Routing

Each node contains one single-head attention unit. Not a full transformer — no feedforward layers, no layer norm, no positional encoding. Just Q, K, V projection. This is the minimum unit that correctly implements the routing operation.

Attention is the correct inductive bias for routing because routing is a similarity computation: how similar is this sample's current trajectory to each branch's characteristic direction. Dot-product attention computes exactly this, explicitly and interpretably. An MLP cannot do this as cleanly — it would need to bake the comparison implicitly into weights.

```
k_L, v_L = branch-specific learned vectors (left child)
k_R, v_R = branch-specific learned vectors (right child)
scores   = softmax([q·k_L, q·k_R])
output   = scores[0]*v_L + scores[1]*v_R
```

The V vectors are the branch vector fields — the actual flow directions for each branch. The attention scores are simultaneously the routing weights and the pruning signal. One mechanism doing three jobs: routing, vector field weighting, and pruning.

3.3 Tree Structure and Natural Termination

The tree is binary, rooted at Gaussian noise at $t=0$. Branches diverge as t progresses. The tree terminates naturally at $t=1$ — samples have arrived at their target modes, the flow is complete, further splitting carries no information. No maximum depth hyperparameter is needed. The flow itself is the depth limit.

The growing threshold enforces coarse-to-fine decomposition naturally. Shallow nodes split on broad distributional differences. Deep nodes only split when fine-grained structure genuinely exists. The tree discovers hierarchy from the data.

4. Self-Calibrating Information-Theoretic Split Criterion

This is the central technical contribution. It replaces every manual threshold with a parameter-free mathematical test derived entirely from the data's own geometric structure.

4.1 The Null Hypothesis

At every node, before splitting, ABFM evaluates one question:

Can the samples currently passing through this node be adequately explained by a single unimodal Gaussian distribution?

If yes — no split. The node's samples are consistent with one mode. If no — split. The samples exceed what a single mode would produce. The test is mathematical, not heuristic. It cannot fire on noise. It cannot be fooled by early trajectory ambiguity.

4.2 The Split Condition

Let X_{node} be the set of sample positions reaching a node at time t . Compute the empirical covariance matrix and extract its largest eigenvalue — the maximum directional variance, the direction along which samples diverge most strongly.

```
lambda_max = largest_eigenvalue_of Cov(X_node)
```

Compare against the theoretical maximum variance expected under a single unimodal Gaussian at time t , derived directly from the flow matching interpolation path $x_t = t \cdot x_1 + (1-t) \cdot x_0$:

$$\text{bound}(t) = (1-t)^2 * \text{sigma_noise}^2 + t^2 * \text{sigma_mode}^2$$

Where sigma_noise^2 is the known Gaussian source variance and sigma_mode^2 is estimated from stage one pretraining. The split fires if and only if:

$$\text{SPLIT if: } \lambda_{\max}(\text{Cov}(X_{\text{node}})) > \text{bound}(t)$$

At $t=0$ the bound is dominated by $(1-t)^2 * \text{sigma_noise}^2$ — large, reflecting pure Gaussian spread. Premature splits on noise are mathematically prevented. As t increases toward 1 the bound tightens to sigma_mode^2 . Only genuine multimodality can exceed this tightened bound. The criterion becomes progressively more sensitive exactly when the flow geometry becomes informative.

4.3 Minimum Sample Guard

Path sampling splits the batch at each node, causing exponential sample decay at deep nodes. Below a minimum count $N_{\min}$, variance estimation is statistically unreliable. Nodes receiving fewer than $N_{\min}$ samples are auto-pruned regardless of variance.

$$\text{if } |X_{\text{node}}| < N_{\min} : \text{prune automatically}$$

4.4 Reviewer Defense

This criterion provides mathematically locked answers to the hardest reviewer questions:

- "How does it know where to split?" — The data tells it. Splits fire when empirical variance mathematically exceeds the single-mode bound.
- "How does it know how many modes there are?" — The data tells it. K is the number of leaves that survive pruning.
- "How did you tune the threshold?" — We did not. There is no threshold to tune. The bound is derived analytically from the flow matching interpolation.

- "Won't it over-split on complex data?" — It cannot. The system cannot split unless the sample batch mathematically violates single-Gaussian variance bounds at that point in the trajectory.

5. Stage One Convergence Gate

The split criterion depends on σ_{mode}^2 estimated from stage one pretraining. A poorly converged estimate produces a wrong bound — catastrophic premature splitting or complete tree stagnation. Stage one must not exit until σ_{mode}^2 has genuinely stabilized. This is verified automatically, not assumed.

5.1 EMA Stability Tracking

At regular evaluation intervals during stage one, compute the largest eigenvalue of the covariance of samples at $t=1$. Maintain an exponential moving average:

```
sigma_EMA(s) = (1-eta) * sigma_EMA(s-1) + eta *
               lambda_max( Cov(X_{t=1}) )
```

Stage one exits automatically when the standard deviation of σ_{EMA} over a sliding window W drops below tolerance ϵ :

```
StdDev( sigma_EMA over window W ) < epsilon -->
freeze sigma_mode^2, activate stage two
```

No manual stage one duration. No fixed step count. The data tells you when the backbone has converged. Stage two always activates on a stable, trustworthy estimate.

5.2 Known Limitation — Heterogeneous Mode Scales

A single global σ_{mode}^2 works correctly when modes have similar variance. When mode scales differ significantly — one tight cluster alongside one diffuse cluster — the global estimate may suppress splits in tight regions. The principled fix is local σ_{mode}^2 per node, estimated from the samples clustering into that branch when stage two activates. This is the one remaining open problem and the natural first extension beyond the proof of concept.

6. Training Procedure

6.1 Two-Stage Curriculum

Training has two stages. This is not a workaround — it is the correct curriculum.

Stage one: standard flow matching, no tree. The shared backbone learns global distribution geometry — where modes are, how samples move. Exits automatically when convergence gate fires and σ_{mode}^2 is frozen.

Stage two: tree activated. The backbone already knows the geometry. Variance signals are immediately meaningful. Branches activate where genuine structure exists. The tree learns on top of an already-competent flow.

One-stage training forces backbone and branches to learn simultaneously on a geometrically uninformed backbone. Variance signals are noisy, splits fire randomly, branches learn incoherent fields. Two stages separates concerns cleanly and is strictly superior.

6.2 Loss Warmup at Stage Two Transition

Abrupt introduction of new loss terms at stage two causes gradient shock that damages the backbone's learned geometry. Both branch loss weights are annealed from zero:

```
lambda(s) = lambda_final * min(1, s / N_warmup)
gamma(s)   = gamma_final   * min(1, s / N_warmup)
```

6.3 Path Sampling

Full tree traversal during training costs $O(2^d)$. Path sampling routes each sample down one greedy path, computing loss only on that path. Cost is $O(d)$. Over many steps every path is visited proportionally to how often samples are routed there — high-traffic branches receive more gradient updates. This is correct behavior, not an approximation. The stochasticity acts as structural dropout, preventing overfitting and forcing backbone generalization.

6.4 Training Objective

$$L = L_{\text{flow}} + \lambda * L_{\text{contrastive}} + \gamma * L_{\text{entropy}}$$

- L_{flow} : standard flow matching loss on the active path. Each branch V vector correctly transports samples toward its target mode.
- $L_{\text{contrastive}}$: penalizes cosine similarity between sibling V vectors. Prevents branch collapse. Computed via small replay buffer storing recent V vectors per branch — siblings are never visited simultaneously under path sampling, buffer solves this with negligible overhead.
- L_{entropy} : rewards high entropy routing early in t , allows low entropy late. Ensures routing decisions migrate toward higher t where geometric structure has emerged. Prevents premature commitment to branches before trajectory geometry is informative.

7. Inference — The Generative Set

7.1 Tree Traversal with Pruning

At inference the full tree is traversed. At each node the information-theoretic criterion is evaluated. Branches where variance does not exceed the single-mode bound are pruned — the node has no genuine multimodal structure to separate. The tree grows only where the distribution warrants it.

7.2 Three Inference Modes

One framework, three inference behaviors selected by the user at query time:

- Deterministic: argmax over surviving leaf attention scores. Single best sample. Fastest.
- Stochastic: sample one leaf proportionally to attention scores. Diverse and principled — weighted by learned mode density, not random uniform sampling.
- Full generative set: return all surviving leaves. Maximum coverage. The complete structured output.

7.3 The Generative Set as Primary Output

The full generative set is a structured collection of diverse high-quality samples — one per surviving leaf — covering all meaningful modes the tree discovered. Set size is adaptive: determined entirely by the distribution complexity, not fixed in advance.

The tree encodes more than samples. It encodes relationships. Modes that share early branches are geometrically related — they diverge late in the trajectory, meaning their distributions are similar at a coarse scale. Modes that diverge at the root are fundamentally distinct. This hierarchical structure is information about the data that no flat generative model can provide.

ABFM does not just generate samples. It generates samples plus geometric structure plus relative mode density plus hierarchical relationships between modes. More information per inference call than any existing flow matching framework.

8. Why This Works — The Strongest Grounds

8.1 The Data Supervises Itself

This is the foundational strength of ABFM. Every structural decision — how many branches, when to split, which samples go where — is derived from the data's own geometric properties. No external labels. No fixed K. No manually tuned thresholds. The practitioner provides data. The framework provides structure.

This is not a philosophical claim. It is a concrete architectural property: the query is the sample's own trajectory position. The split criterion is the sample population's own variance compared to its theoretical single-mode bound. The convergence gate is the data's own variance stabilization. The tree depth is limited by the flow's own termination at $t=1$. Every component reads from the data.

8.2 Attention Is the Correct Inductive Bias

Routing is a similarity computation — how similar is this sample's current position to each branch's characteristic direction. Dot-product attention is precisely a similarity computation. The match between the operation needed and the operation implemented is exact, not approximate. This is why attention is not just a reasonable choice but the correct architectural primitive for this problem.

8.3 The Flow Geometry Is the Supervision Signal

As t increases from 0 to 1, samples transition from undifferentiated Gaussian noise to geometrically distinct positions near their target modes. This transition creates a natural

supervision signal that requires no labels — samples heading toward different modes occupy different regions of space at $t=0.6$ even though they were indistinguishable at $t=0.1$. The framework reads this signal through the query mechanism and routes accordingly. The signal was always there in the flow. ABFM is the first framework to explicitly use it for unsupervised mode discovery.

8.4 Strict Generalization of Standard Flow Matching

ABFM subsumes standard flow matching as a special case. On unimodal distributions the criterion never fires, the tree never branches, and training reduces exactly to standard flow matching with a shared MLP backbone. This provides a theoretical lower bound on performance — ABFM can never perform worse than standard flow matching on unimodal data — and makes the contribution a clean extension rather than a competing approach.

8.5 Adaptive K Without Search

Determining the number of modes K in a distribution is classically hard — it requires model selection, cross-validation, or Bayesian nonparametric methods. ABFM solves this implicitly and for free. K is not selected — it is discovered as the number of leaves that survive pruning. The information-theoretic criterion is the implicit model selection criterion. No separate procedure needed.

9. Comparison to Prior Work

Property	Standard FM	MoE-FM (2025)	ABFM (Ours)
Vector fields	One global	Fixed K flat experts	Adaptive hierarchical tree
Mode discovery	None	None — K fixed manually	Automatic — data-driven
Routing mechanism	None	Separate gating network	Intrinsic attention over V vectors
Threshold tuning	None needed	Manual K selection	None — self-calibrating
Output per call	One sample	One sample	Generative set — adaptive size

Uncertainty handling	Hidden	Hidden	Explicit — full set or stochastic
Training cost	$O(1)$	$O(K)$	$O(d)$ — linear in depth
Labels required	No	No	No — fully unsupervised
Unimodal degeneration	N/A	No clean degeneration	Reduces exactly to standard FM
Hierarchical structure	No	No	Yes — tree encodes mode relationships
Mode scale sensitivity	N/A	N/A	Known limitation — local fix available

Branching Flows (2024): addresses variable-length sequence generation via element-level binary trees controlling how many sequence elements exist at each timestep. Completely orthogonal — their branching is structural and sequential, ours is distributional and routing-level. Potentially composable for variable-length multimodal generation.

10. Target Applications

10.1 Drug Molecule Generation — Primary Target

Drug candidates cluster into fundamentally distinct chemical families — different scaffolds, binding mechanisms, functional groups, pharmacophores. Standard flow matching averages across these families, generating molecules that are chemically mediocre and belong to no clear family. ABFM discovers these families unsupervisedly and generates a structured set covering all of them from one inference call.

The tree encodes chemical relationships: shallow splits separate broad scaffold families, deep splits separate specific functional group variations within families. This structure is scientifically meaningful and directly useful for medicinal chemists exploring chemical space.

10.2 Protein Design

Post-AlphaFold the field needs models that generate structural diversity across protein families, not within one. ABFM's hierarchical decomposition maps naturally onto protein fold families — distantly related folds diverge early in the tree, closely related variants diverge late.

10.3 Materials Science

Novel material design — batteries, catalysts, semiconductors — requires simultaneously exploring structurally distinct material families. Less crowded than molecular generation, strong funding environment, same core multimodality problem. ABFM's generative set output directly serves the diversity requirements of materials discovery pipelines.

11. Known Failure Modes and Mitigations

Failure Mode	Root Cause	Mitigation	Status
Premature splits at $t=0$	Gaussian noise variance exceeds bound	Time-conditioned bound: $(1-t)^2 \sigma_{\text{noise}}^2$ dominates near $t=0$ mathematically	Resolved by design
Batch starvation deep nodes	Path sampling splits batch exponentially	N_{min} guard: auto-prune nodes below minimum sample count	Resolved
Backbone damage at transition	Gradient shock from new loss terms	Loss warmup: λ and γ annealed from zero over N_{warmup} steps	Resolved
Contrastive loss staleness	Siblings never visited simultaneously	Replay buffer: recent V vectors per branch, periodic refresh	Resolved
Trajectory crossover at inference	ODE discretization drift across boundary	Adaptive step size ODE solver	Manageable
Heterogeneous mode scales	Global σ_{mode}^2 suppresses tight clusters	Local σ_{mode}^2 per node — natural extension	Open problem

12. Proof of Concept — Gate-by-Gate Validation Plan

Each gate validates one specific claim. No gate is skipped. A failed gate is diagnosed and fixed before proceeding. Broken foundations are never carried forward.

Gate 1 — Does attention routing work at all?

Target: 1D bimodal. Two Gaussians $N(-4,1)$ and $N(4,1)$ mixed equally. Depth one tree, two leaves. Train two-stage curriculum. Plot attention scores over t from 0 to 1.

Pass condition: scores near 0.5 early in trajectory, sharpen toward 0 and 1 as t approaches 1. Each leaf captures one Gaussian cleanly.

Fail diagnosis: scores stay at 0.5 — increase gamma. Scores collapse to one side always — increase lambda. Scores random — check query spatial sensitivity.

Gate 2 — Does the tree discover modes correctly?

Target: 2D four-cluster Gaussian mixture at $(+3,+3)$, $(+3,-3)$, $(-3,+3)$, $(-3,-3)$. Depth two tree, four leaves. Verify information-theoretic criterion fires at correct nodes and correct t values. Compare against vanilla flow matching baseline visually.

Pass condition: four leaves, one per mode, clean separation. Criterion fires at root and both child nodes, not at leaves. Visual Figure 1 of the paper.

Gate 3 — Does it generalize to nonlinear structure?

Target: pinwheel distribution. Four curved interlocking arms. Tests whether backbone learned global geometry before tree forces localized routing. Trajectory crossovers in dead zones between arms indicate insufficient stage one training.

Gate 4 — Does it work on real data?

Target: QM9 molecular dataset. Standard benchmark. Compare diversity metrics and sample quality against flow matching and MoE-FM baselines. One clean results table. Only attempted after Gates 1-3 pass cleanly.

13. One Remaining Open Problem

Local σ_{mode}^2 for heterogeneous mode scales. When target distributions contain modes of vastly different variance — one tight cluster, one diffuse cluster — a single global σ_{mode}^2 from stage one underestimates tight cluster expected variance, potentially suppressing splits there.

The solution is local: each node maintains its own σ_{mode}^2 estimated from samples initially clustering into that branch when stage two activates. Mathematically straightforward, practically important for real molecular datasets. This is the natural first extension beyond the proof of concept and a clean future work contribution.

14. Paper Narrative and Framing

The paper tells one clean story:

1. Standard flow matching fails on multimodal distributions because one vector field cannot serve multiple transport geometries.
2. Existing solutions require external knowledge — fixed K, labels, manual thresholds — that practitioners often do not have.
3. The sample's own trajectory through the flow carries all the information needed to discover modes unsupervisedly.
4. ABFM reads that signal through intrinsic attention routing and an information-theoretic split criterion derived entirely from the data.
5. The result is a self-calibrating framework that outputs a structured generative set covering all discovered modes with no manual tuning.
6. It subsumes standard flow matching, outperforms MoE-FM on multimodal benchmarks, and provides hierarchical mode structure as a byproduct.

The abstract hook: not 'we built a heuristic tree on top of flow matching' but 'we present a self-calibrating framework that mathematically senses multimodality to dynamically branch continuous velocity fields — requiring no labels, no fixed K, and no manual thresholds.'

Framework complete. All components specified. All known failure modes addressed except one open problem (local σ_{mode}^2) clearly scoped for future work. The data supervises itself — this is the ground everything stands on.

Next step: Gate 1 — implement vanilla flow matching, then add single-node attention routing.

Two mixed Gaussians. One routing node. One binary question: does attention separate them?